

YOLOv8-Seg 螺丝刀识别 Demo 用户手册

1. 手册目标

本文档用于指导在 `leju-kuavo-challenge-cup-2026-ChengLihan` 项目中完成 YOLOv8 分割模型的快速 demo，包括：

1. 获取开源数据集。
2. 理解和使用 YOLO 分割数据集格式。
3. 标注或补充自己的 Scene2 数据。
4. 使用 `yolov8n-seg.pt` 进行训练。
5. 预测测试集和 Scene2 截图。
6. 查看训练结束后的各项指标。
7. 判断模型是否适合接入机械臂抓取流程。
8. 将训练好的模型部署到 Docker 中进行推理。

当前阶段只做螺丝刀 `screwdriver` 的快速 demo，目标是先验证：

图像输入

- YOLOv8-seg 检测螺丝刀
- 输出 bbox、mask、confidence
- 计算 mask 中心点
- 后续接 depth 和 camera_info 得到 3D 坐标

当前不要求一次性完成 `pipe_clamp`、`pipe_fitting` 和分拣箱识别，后续可以逐步补数据扩展。

2. 推荐模型选择

当前使用：

`yolov8n-seg.pt`

理由：

1. `n` 模型体积小，训练和推理速度快。
2. `seg` 模型可以输出实例分割 mask。
3. mask 可以用于后续取深度中值、计算物体中心、计算 PCA 主方向。
4. 比普通检测框更适合机械臂抓取定位。
5. 当前 demo 只检测螺丝刀，用 `yolov8n-seg.pt` 足够。

暂时不建议使用：

```

yolov8n.pt      # 只有检测框，没有 mask
yolov8s-seg.pt  # 精度可能更高，但训练和推理更慢，等 demo 跑通后再换
yolov8-pose     # 需要关键点标注，当前阶段成本太高
yolov8-obb      # 可用于姿态角，但当前先用 mask PCA 替代

```

后续升级路线：

```

第一阶段：YOLOv8n-seg + 开源 screwdriver 数据集
第二阶段：补充 30~100 张 Scene2 螺丝刀图像进行 fine-tune
第三阶段：加入 pipe_clamp、pipe_fitting
第四阶段：加入 purple_bin、blue_bin、orange_bin
第五阶段：根据抓取效果决定是否升级 YOLOv8s-seg 或增加 OBB

```

3. 项目目录规划

进入项目仓库：

```
cd ~/leju-kuavo-challenge-cup-2026-ChengLihan
```

推荐目录结构：

```

leju-kuavo-challenge-cup-2026-ChengLihan/
├── .venv_yolo_demo/           # 本机训练环境
├── datasets/                  # 自己整理或下载的数据集
├── Workinghands-4/           # Roboflow 下载的数据集，目录名可能不同
├── models/
│   └── yolo/
│       └── yolov8n_seg_screwdriver_demo.pt # 最终部署用模型
├── runs/
│   ├── segment/
│   └── scene2_demo/
├── src/
│   ├── challenge_cup_task_template/
│   └── scripts/
│       └── screwdriver_demo_detector.py

```

创建必要目录：

```
mkdir -p datasets
mkdir -p models/yolo
mkdir -p runs/scene2_demo
```

4. 本机训练环境安装

本项目采用：

本机训练 YOLO
Docker 内只部署推理

不要一开始就在 Docker 里训练，避免 PyTorch、CUDA、ROS2 依赖互相污染。

在仓库根目录创建虚拟环境：

```
cd ~/leju-kuavo-challenge-cup-2026-ChengLihan

python3 -m venv .venv_yolo_demo
source .venv_yolo_demo/bin/activate

pip install -U pip
pip install ultralytics roboflow opencv-python numpy matplotlib pyyaml pandas
tensorboard
```

检查 YOLO：

```
yolo checks
```

检查 GPU：

```
python - <<'EOF'
import torch
print("CUDA available:", torch.cuda.is_available())
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU
only")
EOF
```

如果显示：

```
CUDA available: True
NVIDIA GeForce RTX 4060 Laptop GPU
```

说明可以使用 GPU 训练。

如果没有 CUDA，也可以先用 CPU 训练 demo，只是速度会慢。

5. 获取开源数据集

5.1 推荐数据集

当前 demo 推荐使用 Roboflow Universe 上的工具类分割数据集 `Workinghands`。

该数据集包含：

```
screwdriver
hammer
wrench
pliers
```

当前只使用其中的 `screwdriver` 作为 demo 类别，其他类别保留，但推理时过滤掉。

5.2 下载 Roboflow 数据集

先注册 Roboflow 并获取 API Key。

在仓库根目录创建下载脚本：

```
nano download_workinghands.py
```

写入：

```
from roboflow import Roboflow

rf = Roboflow(api_key="替换成你的_ROBOFLOW_API_KEY")

project = rf.workspace("mechanical-tools").project("workinghands")

dataset = project.version(2).download("yolov8")
```

```
print("Downloaded to:", dataset.location)
```

运行：

```
source .venv_yolo_demo/bin/activate  
python download_workinghands.py
```

如果 `version(2)` 下载失败，则将脚本改成：

```
dataset = project.version(1).download("yolov8")
```

再重新运行。

下载后可能出现类似目录：

```
Workinghands-2/  
Workinghands-4/
```

实际目录名以本机为准。

6. 检查数据集结构

进入下载目录查看：

```
ls
```

典型 Roboflow YOLOv8 数据集结构：

```
Workinghands-4/  
  train/  
    images/  
    labels/  
  valid/  
    images/  
    labels/  
  test/  
    images/
```

```
labels/  
data.yaml
```

查看 `data.yaml` :

```
cat Workinghands-4/data.yaml
```

内容应类似：

```
train: ../train/images  
val: ../valid/images  
test: ../test/images  
  
nc: 4  
names:  
  - hammer  
  - pliers  
  - screwdriver  
  - wrench
```

或者：

```
names:  
  0: hammer  
  1: pliers  
  2: screwdriver  
  3: wrench
```

注意：

必须确认 `screwdriver` 的类别名存在。
不要自己猜 `class id`。
推理时应使用 `result.names` 读取类别名，再过滤 `screwdriver`。

7. YOLOv8-Seg 标注格式说明

YOLOv8 segmentation 的标签文件是 `.txt`，每张图片对应一个同名 `.txt` 标签文件。

例如：

```
images/train/0001.jpg  
labels/train/0001.txt
```

每一行表示一个实例：

```
class_id x1 y1 x2 y2 x3 y3 x4 y4 ...
```

说明：

```
class_id : 类别编号  
x1 y1 x2 y2 ... : polygon 多边形点坐标  
坐标全部归一化到 0~1  
一行代表一个物体实例  
一张图有多个物体，就有多行
```

示例：

```
2 0.312 0.455 0.330 0.460 0.350 0.480 0.340 0.500
```

如果 2 对应 `screwdriver`，则这一行表示一个螺丝刀实例的 mask polygon。

8. 后续补充 Scene2 数据的方法

开源数据集只能用于快速 demo。为了让模型真正适配比赛仿真环境，后续必须补充 Scene2 图像。

8.1 采集 Scene2 图片

从仿真中采集：

```
头部相机图像  
左腕相机图像  
右腕相机图像
```

建议第一批只采：

```
30~50 张含有螺丝刀的 Scene2 截图
```

后续扩展到：

100~300 张螺丝刀图像
再加入 pipe_clamp 和 pipe_fitting
再加入分拣箱

8.2 推荐标注工具

可以使用：

CVAT
Labelme
Roboflow Annotate

标注任务类型选择：

Instance Segmentation

导出格式选择：

Ultralytics YOLO Segmentation
YOLOv8 Segmentation

8.3 标注要求

螺丝刀标注要求：

1. 使用 polygon/mask 贴合螺丝刀可见区域。
2. 红色柄部和金属杆都要标出。
3. 遮挡部分不要凭空补全，只标可见部分。
4. 不要把阴影标进去。
5. 不要把桌面、夹爪、箱子误标为螺丝刀。

如果只做一类螺丝刀，Scene2 微调数据集可以这样组织：

```
datasets/scene2_screwdriver_seg/  
  images/  
    train/  
    val/  
  labels/  
    train/
```



```
val/  
data.yaml
```

`data.yaml` :

```
path: datasets/scene2_screwdriver_seg  
train: images/train  
val: images/val  
  
names:  
  0: screwdriver
```

9. 训练命令

9.1 快速 debug 训练

第一轮只跑 20 epoch，目的是确认数据、环境、路径没问题。

```
source .venv_yolo_demo/bin/activate  
  
yolo task=segment mode=train  
  model=yolov8n-seg.pt  
  data=Workinghands-4/data.yaml  
  imgsz=640  
  epochs=20  
  batch=8  
  device=0  
  project=runs/scene2_demo  
  name=yolov8n_seg_workinghands_debug
```

如果没有 GPU：

```
device=cpu
```

如果数据集目录不是 `Workinghands-4`，请改成实际目录，例如：

```
data=Workinghands-2/data.yaml
```

9.2 正式 demo 训练

debug 没问题后，跑 50 epoch：

```
source .venv_yolo_demo/bin/activate

yolo task=segment mode=train
  model=yolov8n-seg.pt
  data=Workinghands-4/data.yaml
  imgsz=640
  epochs=50
  batch=16
  patience=15
  device=0
  project=runs/scene2_demo
  name=yolov8n_seg_workinghands_v1
```

如果显存不够：

```
batch=8
```

如果想更保守：

```
epochs=30
batch=8
```

9.3 基于已有 demo 模型微调 Scene2 螺丝刀

当补充了 Scene2 螺丝刀数据后，用当前模型继续训练：

```
source .venv_yolo_demo/bin/activate

yolo task=segment mode=train
  model=models/yolo/yolov8n_seg_screwdriver_demo.pt
  data=datasets/scene2_screwdriver_seg/data.yaml
  imgsz=640
  epochs=20
  batch=8
  device=0
  project=runs/scene2_demo
  name=yolov8n_seg_scene2_screwdriver_ft
```

训练结束后复制最优权重：

```
cp runs/scene2_demo/yolov8n_seg_scene2_screwdriver_ft/weights/best.pt
models/yolo/yolov8n_seg_screwdriver_demo.pt
```

如果实际保存路径不同，不要手猜路径，用 `find` 查找。

10. 预测命令

10.1 对测试集预测

```
source .venv_yolo_demo/bin/activate

yolo task=segment mode=predict
model=models/yolo/yolov8n_seg_screwdriver_demo.pt
source=Workinghands-4/test/images
imgsz=640
conf=0.25
save=True
```

10.2 保存预测标签和置信度

```
yolo task=segment mode=predict
model=models/yolo/yolov8n_seg_screwdriver_demo.pt
source=Workinghands-4/test/images
imgsz=640
conf=0.25
save=True
save_txt=True
save_conf=True
```

输出目录通常为：

```
runs/segment/predict
runs/segment/predict2
runs/segment/predict3
```

查看最新预测目录：

```
find runs/segment -maxdepth 1 -type d -name "predict*" | sort
```

打开预测结果：

```
xdg-open runs/segment/predict
```

10.3 对 Scene2 截图预测

将 Scene2 截图保存为：

```
test_scene2.png
```

然后运行：

```
yolo task=segment mode=predict  
  model=models/yolo/yolov8n_seg_screwdriver_demo.pt  
  source=test_scene2.png  
  imgsz=640  
  conf=0.15  
  save=True
```

说明：

Scene2 仿真图像和开源数据集存在域差异，第一次测试时 conf 可以降低到 0.15。
如果误检很多，再逐步提高到 0.25 或 0.35。
如果完全检测不到，说明需要补充 Scene2 螺丝刀数据进行 fine-tune。

11. 训练结束后如何系统查看结果

11.1 不要猜路径，先找 best.pt

每次训练后都执行：

```
cd ~/leju-kuavo-challenge-cup-2026-ChengLihan  
  
find runs -path "*weights/best.pt" -type f
```

输出可能类似：

```
runs/segment/runs/scene2_demo/yolov8n_seg_workinghands_demo-2/weights/best.pt
```

注意：

不同训练任务、不同 project/name 设置可能导致保存路径不同。
不要凭记忆写路径。
必须用 find 找真实 best.pt。

设置变量：

```
BEST=$(find runs -path "**/weights/best.pt" -type f | sort | tail -n 1)
RUN=$(dirname "$(dirname "$BEST)")

echo "BEST = $BEST"
echo "RUN  = $RUN"
```

查看 run 目录：

```
ls -lah "$RUN"
```

12. YOLO run 目录里每个文件怎么看

一个完整训练目录通常包含：

```
args.yaml
results.csv
results.png
confusion_matrix.png
confusion_matrix_normalized.png
labels.jpg
labels_correlogram.jpg
train_batch*.jpg
val_batch*_labels.jpg
val_batch*_pred.jpg
BoxP_curve.png
BoxR_curve.png
BoxF1_curve.png
BoxPR_curve.png
MaskP_curve.png
MaskR_curve.png
```

```
MaskF1_curve.png
MaskPR_curve.png
weights/
```

重点文件：

```
args.yaml
    记录本次训练的所有参数。

results.csv
    每个 epoch 的 loss、precision、recall、mAP、learning rate。

results.png
    训练曲线总览。

confusion_matrix.png
    类别混淆矩阵，用于查看类别之间是否互相误识别。

val_batch*_pred.jpg
    验证集预测可视化结果，必须人工查看 mask 是否贴合物体。

MaskF1_curve.png
    mask 分割任务的 F1 曲线。

MaskPR_curve.png
    mask 分割任务的 precision-recall 曲线。

weights/best.pt
    验证集指标最好的模型，一般用于部署。

weights/last.pt
    最后一轮模型，一般用于继续训练或排查。
```

打开 run 目录：

```
xdg-open "$RUN"
```

打开主要曲线：

```
xdg-open "$RUN/results.png"
xdg-open "$RUN/confusion_matrix.png"
xdg-open "$RUN/MaskF1_curve.png"
```

13. 查看训练参数 args.yaml

查看本次训练参数：

```
cat "$RUN/args.yaml"
```

或者用 Python 格式化查看：

```
python - <<EOF
import yaml
from pprint import pprint

with open("$RUN/args.yaml", "r") as f:
    args = yaml.safe_load(f)

pprint(args)
EOF
```

重点关注字段：

```
model
    初始权重，例如 yolov8n-seg.pt 或某个 best.pt。

data
    数据集 yaml 路径。

epochs
    训练总轮数。

batch
    batch size。

imgsz
    输入图像尺寸。

device
    GPU 或 CPU。

project
    保存目录。

name
    run 名字。
```

patience
early stopping 等待轮数。

optimizer
优化器。

lr0
初始学习率。

augment 相关参数
数据增强设置。

14. 查看每轮训练指标 results.csv

查看最后几轮：

```
tail -n 5 "$RUN/results.csv"
```

更清晰地查看表格：

```
column -s, -t "$RUN/results.csv" | less -S
```

用 Python 查看列名和最后一轮：

```
python - <<EOF
import pandas as pd

df = pd.read_csv("$RUN/results.csv")
df.columns = [c.strip() for c in df.columns]

print("Columns:")
for c in df.columns:
    print(c)

print("\nLast epoch:")
print(df.tail(1).T)
EOF
```

常见关键列：

epoch

train/box_loss
train/seg_loss
train/cls_loss
train/dfl_loss

val/box_loss
val/seg_loss
val/cls_loss
val/dfl_loss

metrics/precision(B)
metrics/recall(B)
metrics/mAP50(B)
metrics/mAP50-95(B)

metrics/precision(M)
metrics/recall(M)
metrics/mAP50(M)
metrics/mAP50-95(M)

其中：

B = Box
表示检测框指标。

M = Mask
表示分割 mask 指标。

本项目后续要接 depth 和机械臂抓取，所以更关注：

metrics/precision(M)
metrics/recall(M)
metrics/mAP50(M)
metrics/mAP50-95(M)

15. 自动找最佳 epoch

用 Mask mAP50-95 找最优 epoch：

```
python - <<EOF
import pandas as pd

df = pd.read_csv("$RUN/results.csv")
df.columns = [c.strip() for c in df.columns]

target_col = None
for c in df.columns:
    if "mAP50-95" in c and "(M)" in c:
        target_col = c
        break

if target_col is None:
    print("No Mask mAP50-95 column found.")
else:
    idx = df[target_col].idxmax()
    print("Best epoch by", target_col)
    print(df.loc[idx].T)
EOF
```

查看最后一轮所有 metrics：

```
python - <<EOF
import pandas as pd

df = pd.read_csv("$RUN/results.csv")
df.columns = [c.strip() for c in df.columns]

cols = [c for c in df.columns if "metrics" in c or c == "epoch"]
print(df[cols].tail(1).T)
EOF
```

16. 各项指标如何判断

16.1 Precision

含义：

模型预测出来的目标里面，有多少是真的。

判断：

Precision 高：
误检少。
Precision 低：
容易把其他物体误识别成目标。

对本项目的影响：

如果 screwdriver precision 低，机械臂可能去抓错物体。

16.2 Recall

含义：

真实存在的目标中，有多少被模型找到了。

判断：

Recall 高：
漏检少。
Recall 低：
图里有螺丝刀，但模型经常看不见。

对本项目的影响：

如果 screwdriver recall 低，机器人可能找不到螺丝刀，任务卡死。

16.3 mAP50

含义：

IoU 阈值为 0.50 时的平均精度。

判断：

mAP50 高：
模型大致能检测或分割到目标。

特点：

mAP50 是比较宽松的指标。

16.4 mAP50-95

含义：

IoU 阈值从 0.50 到 0.95 的平均 mAP。

判断：

mAP50-95 高：
定位和分割更精确。

特点：

mAP50-95 比 mAP50 更严格。

对本项目的影响：

机械臂抓取需要 mask 中心和 mask 内深度，因此 mAP50-95(M) 比 mAP50(M) 更有参考价值。

16.5 Box 指标和 Mask 指标

Box 指标：
检测框是否准确。

Mask 指标：
实例分割轮廓是否准确。

本项目优先看：

Mask Precision
Mask Recall
Mask mAP50
Mask mAP50-95

因为后续要用 mask 来：

1. 取 mask 中心。
2. 取 mask 内 depth 中值。
3. 过滤背景深度。
4. 计算物体主方向。

17. 如何判断模型是否适合机械臂抓取

不要只看 mAP，要结合可视化结果判断。

17.1 必须人工查看预测图

打开：

```
xdg-open "$RUN"
```

重点看：

```
val_batch*_pred.jpg  
runs/segment/predict*/预测图
```

检查：

1. 螺丝刀是否被检测到。
2. mask 是否覆盖整个螺丝刀。
3. mask 是否漏掉金属杆或手柄。
4. mask 是否包含桌面阴影。
5. 是否把锤子、钳子、扳手误识别为 screwdriver。
6. 多个螺丝刀是否能分别分割成多个实例。

17.2 机械臂抓取更关注什么

对抓取来说，最关键不是框漂不漂，而是：

1. mask 中心点是否稳定。
2. mask 内 depth 是否能取到有效中值。
3. mask 是否不包含大量背景。
4. mask 是否能反映物体大致长轴方向。
5. Scene2 仿真图中是否能泛化。

17.3 当前 demo 的合格标准

快速 demo 阶段：

Workinghands 测试集能识别 screwdriver。
能输出 mask。
推理速度正常。
能运行 predict。
能生成可视化结果。

Scene2 接入前：

Scene2 截图中至少能检测出螺丝刀。
mask 中心大致落在物体上。
误检数量可控。

可用于机械臂抓取前：

Scene2 多张图中都能稳定检测。
mask 内 depth 有效。
3D 坐标稳定。
不同视角下不频繁漏检。

18. TensorBoard 查看训练过程

安装：

```
pip install tensorboard
```

启动：

```
tensorboard --logdir runs --port 6006
```

浏览器打开：

```
http://localhost:6006
```

TensorBoard 适合查看：

loss 曲线
mAP 曲线
不同训练 run 的对比
学习率变化

19. 查看模型信息

查看模型类别、参数量和结构信息：

```
python - <<EOF
from ultralytics import YOLO

model = YOLO("$BEST")
print("names:", model.names)
model.info()
EOF
```

典型输出包含：

类别 names
模型层数
参数量
GFLOPs

查看权重大小：

```
ls -lh "$RUN/weights"
```

通常：

best.pt
部署和推理优先使用。

last.pt
继续训练或恢复训练时使用。

20. 固定保存部署模型

每次训练后找到最好的 `best.pt`：

```
BEST=$(find runs -path "**/weights/best.pt" -type f | sort | tail -n 1)
echo "$BEST"
```

复制到统一部署路径：

```
mkdir -p models/yolo

cp "$BEST" models/yolo/yolov8n_seg_screwdriver_demo.pt
```

后续所有脚本统一使用：

```
models/yolo/yolov8n_seg_screwdriver_demo.pt
```

这样可以避免路径混乱。

21. 最小推理脚本

脚本路径：

```
src/challenge_cup_task_template/scripts/screwdriver_demo_detector.py
```

脚本功能：

1. 加载 `models/yolo/yolov8n_seg_screwdriver_demo.pt`。
2. 输入一张图片。
3. 使用 YOLOv8-seg 推理。
4. 只保留 `screwdriver` 类别。
5. 输出 `bbox`、`mask`、`confidence`、`mask center`。
6. 保存可视化图片。

脚本内容：

```
import cv2
import numpy as np
```



```

from ultralytics import YOLO

class ScrewdriverDemoDetector:
    def __init__(self, model_path, conf=0.25, imgsz=640, device=0):
        self.model = YOLO(model_path)
        self.conf = conf
        self.imgsz = imgsz
        self.device = device

    def detect_screwdriver(self, bgr_image):
        results = self.model.predict(
            source=bgr_image,
            imgsz=self.imgsz,
            conf=self.conf,
            device=self.device,
            retina_masks=True,
            verbose=False,
        )

        result = results[0]
        detections = []

        if result.names is None or result.masks is None:
            return detections

        names = result.names
        boxes = result.boxes
        masks = result.masks.data.cpu().numpy()

        for i in range(len(boxes)):
            cls_id = int(boxes.cls[i].item())
            cls_name = names[cls_id]
            score = float(boxes.conf[i].item())

            if cls_name != "screwdriver":
                continue

            mask = (masks[i] > 0.5).astype(np.uint8)
            xyxy = boxes.xyxy[i].cpu().numpy().tolist()
            center = self.get_mask_center(mask)

            detections.append({
                "class_id": cls_id,
                "class_name": cls_name,
                "confidence": score,
                "bbox_xyxy": xyxy,
                "mask": mask,
            })

```

```

        "center_uv": center,
    })

    return detections

@staticmethod
def get_mask_center(mask):
    ys, xs = np.where(mask > 0)
    if len(xs) == 0:
        return None
    u = float(np.mean(xs))
    v = float(np.mean(ys))
    return u, v

if __name__ == "__main__":
    model_path = "models/yolo/yolov8n_seg_screwdriver_demo.pt"
    image_path = "test_scene2.png"

    detector = ScrewdriverDemoDetector(
        model_path=model_path,
        conf=0.25,
        imgsz=640,
        device=0,
    )

    img = cv2.imread(image_path)
    if img is None:
        raise FileNotFoundError(f"Cannot read image: {image_path}")

    dets = detector.detect_screwdriver(img)
    print("detections:", len(dets))

    for d in dets:
        print(d["class_name"], d["confidence"], d["center_uv"], d["bbox_xyxy"])

        mask = d["mask"]
        img[mask > 0] = img[mask > 0] * 0.5 + np.array([0, 0, 255]) * 0.5

        if d["center_uv"] is not None:
            u, v = d["center_uv"]
            cv2.circle(img, (int(u), int(v)), 5, (0, 255, 0), -1)

    cv2.imwrite("screwdriver_demo_result.png", img)
    print("saved to screwdriver_demo_result.png")

```

运行：

```
source .venv_yolo_demo/bin/activate
python src/challenge_cup_task_template/scripts/screwdriver_demo_detector.py
```

22. 接 depth 的核心函数

YOLO 输出 mask 后，下一步接深度图。

```
import numpy as np

def get_mask_median_depth(depth_img, mask):
    valid = depth_img[mask > 0]
    valid = valid[np.isfinite(valid)]
    valid = valid[valid > 0]

    if len(valid) < 30:
        return None

    low = np.percentile(valid, 10)
    high = np.percentile(valid, 90)
    valid = valid[(valid >= low) & (valid <= high)]

    return float(np.median(valid))

def pixel_to_camera_xyz(u, v, z, K):
    fx = K[0]
    fy = K[4]
    cx = K[2]
    cy = K[5]

    x = (u - cx) * z / fx
    y = (v - cy) * z / fy

    return np.array([x, y, z], dtype=np.float32)
```

注意：

depth 图可能是 uint16 毫米，也可能是 float32 米。
必须先打印 dtype 和数值范围。

检查 depth 单位：

```
print(depth_img.dtype, np.nanmin(depth_img), np.nanmax(depth_img))
```

如果是毫米：

```
depth_m = depth_img.astype(np.float32) / 1000.0
```

如果已经是米：

```
depth_m = depth_img.astype(np.float32)
```

23. Docker 内部署检查

训练在本机完成，Docker 内只做部署推理。

进入 Docker 后，进入项目路径，例如：

```
cd /root/kuavo_ws/src/leju-kuavo-challenge-cup-2026-ChengLihan
```

或者：

```
cd /root/project
```

安装推理依赖：

```
pip3 install ultralytics opencv-python numpy
```

如果出现 `externally-managed-environment`：

```
pip3 install ultralytics opencv-python numpy --break-system-packages
```

更稳的方式是在 Docker 内建推理环境：

```
python3 -m venv /root/yolo_runtime  
source /root/yolo_runtime/bin/activate
```

```
pip install -U pip
pip install ultralytics opencv-python numpy
```

检查模型能否加载：

```
python3 - <<'EOF'
from ultralytics import YOLO

model = YOLO("models/yolo/yolov8n_seg_screwdriver_demo.pt")
print("model loaded")
print(model.names)
EOF
```

检查 Docker 内 GPU：

```
nvidia-smi

python3 - <<'EOF'
import torch
print("CUDA available:", torch.cuda.is_available())
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU
only")
EOF
```

如果 CUDA 不可用，先允许 CPU 推理跑通。

24. 常见问题排查

问题 1：FileNotFoundError，找不到 best.pt

原因：

模型路径写错。
训练输出目录和命令中预期目录不一致。

解决：

```
find runs -path "**/weights/best.pt" -type f
```

然后复制真实路径。

推荐永久解决：

```
BEST=$(find runs -path "*/weights/best.pt" -type f | sort | tail -n 1)
mkdir -p models/yolo
cp "$BEST" models/yolo/yolov8n_seg_screwdriver_demo.pt
```

以后都用：

```
models/yolo/yolov8n_seg_screwdriver_demo.pt
```

问题 2：Workinghands-2 和 Workinghands-4 路径不一致

原因：

Roboflow 多次下载会生成不同版本目录。

解决：

```
ls
find . -name "data.yaml"
```

训练时把 `data=` 改成真实路径。

问题 3：预测时没有检测到螺丝刀

解决：

1. 降低 conf, 例如 0.35 → 0.25 → 0.15。
2. 查看预测图, 判断是漏检还是类别错分。
3. 用 30~50 张 Scene2 图像 fine-tune。
4. 检查 data.yaml 类别名是否正确。
5. 检查图片是否过暗、过小、视角过偏。

问题 4：只出现 bbox, 没有明显 mask

解决：

1. 确认使用的是 yolov8n-seg.pt, 不是 yolov8n.pt。
2. 确认 task=segment。

3. 确认数据集是 segmentation 格式。
4. 推理脚本里使用 result.masks。

问题 5：训练指标很高，但 Scene2 效果差

原因：

开源数据集和仿真环境存在 domain gap。

解决：

必须补 Scene2 图像进行微调。
先补 30~50 张螺丝刀。
再补 100~300 张。
最后补其他类别。

25. 每次训练后的标准检查流程

每次 YOLO 训练完成后，都按下面流程检查：

```
# 1. 找到最新 best.pt
BEST=$(find runs -path "**/weights/best.pt" -type f | sort | tail -n 1)
RUN=$(dirname "$(dirname "$BEST")")
echo "BEST = $BEST"
echo "RUN = $RUN"

# 2. 查看 run 目录
ls -lah "$RUN"

# 3. 查看训练参数
cat "$RUN/args.yaml"

# 4. 查看最后几轮指标
tail -n 5 "$RUN/results.csv"

# 5. 打开曲线图和预测图
xdg-open "$RUN/results.png"
xdg-open "$RUN/confusion_matrix.png"
xdg-open "$RUN"

# 6. 查看模型类别和参数
```

```
python - <<EOF
from ultralytics import YOLO
model = YOLO("$BEST")
print(model.names)
model.info()
EOF

# 7. 复制为部署模型
mkdir -p models/yolo
cp "$BEST" models/yolo/yolov8n_seg_screwdriver_demo.pt

# 8. 用部署路径重新预测
yolo task=segment mode=predict
    model=models/yolo/yolov8n_seg_screwdriver_demo.pt
    source=Workinghands-4/test/images
    imgsz=640
    conf=0.25
    save=True
    save_txt=True
    save_conf=True
```

26. 当前阶段最终目标

当前阶段完成以下内容即可：

1. 能成功训练 YOLOv8n-seg。
2. 能下载并使用 Workinghands 数据集。
3. 能在测试集上检测 screwdriver。
4. 能输出 mask。
5. 能用 Python 脚本过滤 screwdriver。
6. 能得到 mask center。
7. 能把 best.pt 固定复制到 models/yolo。
8. 能在 Docker 中加载同一个模型。
9. 能在 Scene2 截图上初步测试。

完成后，下一阶段再做：

1. 接入 ROS2 图像话题。
2. 接 depth 图和 camera_info。
3. 使用 mask 内 depth 中值计算 3D 坐标。
4. 使用 TF 转到 base_link。

5. 控制机械臂移动到目标上方。
6. 用腕部相机进行二次校准。

27. 一句话总结

当前 YOLO demo 的核心流程是：

获取 Workinghands 开源分割数据集
→ 使用 yolov8n-seg.pt 快速训练 screwdriver demo
→ 查看 Mask 指标和预测图
→ 固定 best.pt 到 models/yolo
→ Docker 内加载模型推理
→ 后续用 Scene2 少量数据 fine-tune
→ 接入 RGB-D 和机械臂抓取流程